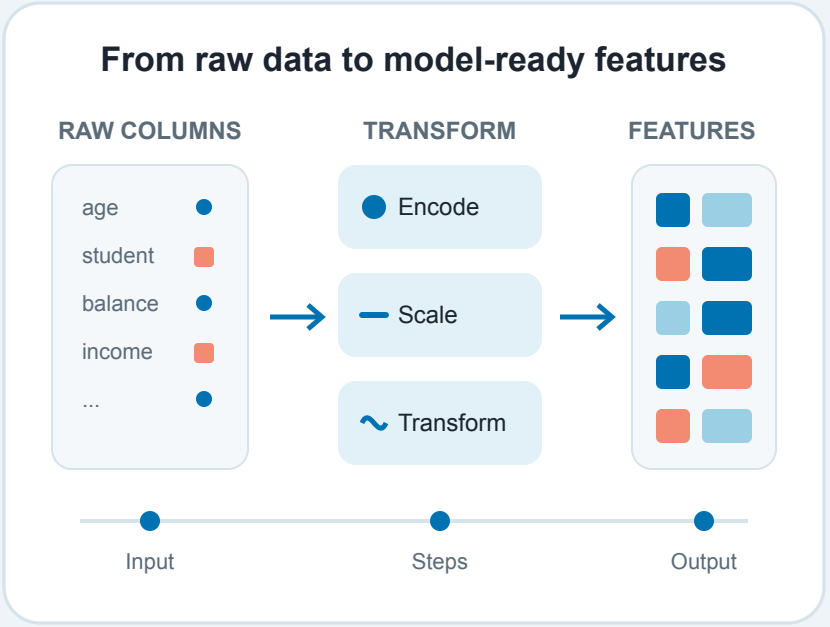
DATA 201 · WEEK 7

Feature engineering and pipelines

Encode, scale, and transform the columns a model learns from



Rebin Muhammad
Montgomery College

WHERE WE ARE

From choosing a model to preparing its input

Last week

Choose

Overfitting, cross-validation, and grid search. The question was which model and which settings.

This week

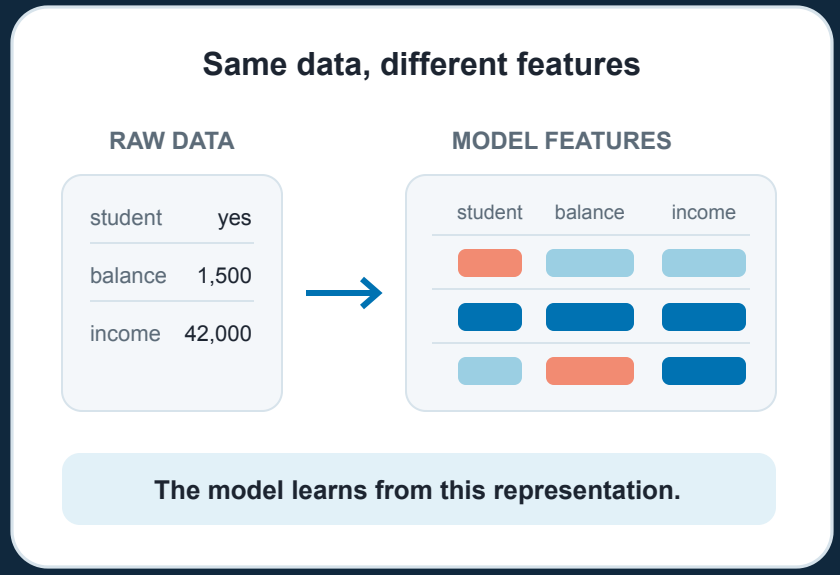
Represent

Encode categories, scale columns, transform shapes, and chain the steps in one pipeline.

THE IDEA

We choose the features a model learns from.

Encoding, scaling, and transforming data changes what the model can learn.



WHY THE REPRESENTATION MATTERS

The same data can teach a model different things

Scale

Centimeters or inches

KNN uses distance. A feature with a larger numerical scale can dominate that distance.

Shape

A curve, not a straight line

A square, a log, or bins can help a model represent a nonlinear relationship.

Categories

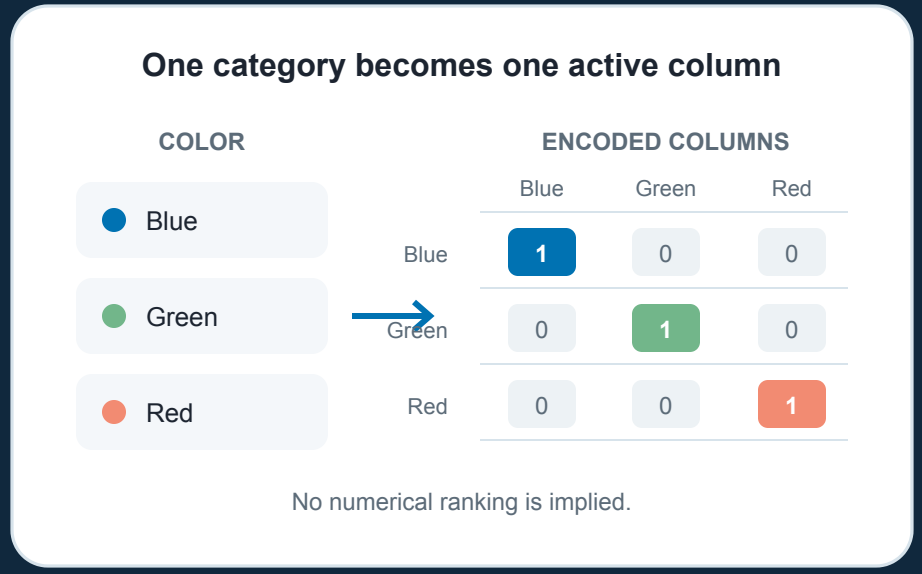
“Master’s” or “Private”

Encode category names as numbers without implying an order that isn’t there.

PART 1

Encoding

Turn category labels into columns a model can use.



INPUTS AND TARGETS

The target determines the task

Numeric inputs

Age · hours · balance

Numbers have order, and differences between values are meaningful.

Categorical inputs

Workclass · occupation

Values are labels. “Private” is not greater than “State-gov.”

What are we trying to predict?

Predict an amount → Regression

Example: predict someone’s income in dollars.

Predict a category → Classification

Example: predict whether income is above $50,000.

ADULT INCOME

Five people, seven columns

1994 U.S. census: does this worker earn more than $50,000?

Blue headers are continuous. Navy headers are categories. Orange is the target.

Age	Work class	Education	Gender	Hours	Occupation	Income
39	State-gov	Bachelors	Male	40	Adm-clerical	<=50K
50	Self-emp-not-inc	Bachelors	Male	13	Exec-managerial	<=50K
38	Private	HS-grad	Male	40	Handlers-cleaners	<=50K
28	Private	Bachelors	Female	40	Prof-specialty	<=50K
52	Self-emp-not-inc	HS-grad	Male	45	Exec-managerial	>50K

Age and hours are numeric inputs. Work class, education, sex, and occupation are categories. Income is the target.

ONE-HOT ENCODING

One column per category

Four names, four columns. One switch each.

The word	Government	Private	Self-employed	Self-emp. inc.
Government	**1**	**0**	**0**	**0**
Private	**0**	**1**	**0**	**0**
Self-employed	**0**	**0**	**1**	**0**
Self-emp. inc.	**0**	**0**	**0**	**1**

Each row gets a 1 for its category and 0s for the others.

WHY NOT 0, 1, 2, 3?

Integer codes invent an order

Integer codes

0, 1, 2, 3

These codes imply an order and distance between categories that may not exist.

One-hot encoding

One indicator per category

The model can learn a separate effect for each category.

Keep all k

scikit-learn default

OneHotEncoder keeps every category unless you choose a drop option.

Drop one

k − 1 indicators

The omitted category becomes the reference. This can avoid redundancy in models with an intercept.

FOUR PITFALLS

Four encoding mistakes to avoid

Target leakage

Keep income out of X

Encoded income belongs in y. If it stays in X, the model has already been given the answer.

Inconsistent labels

“Private” and “private”

The encoder may treat these as different categories. Check the values and standardize labels before encoding.

Numeric category codes

Workclass stored as 0, 1, 2…

These are labels, not measurements. Treat the column as categorical and encode it accordingly.

Numeric columns

`get_dummies` may leave them alone

Tell pandas which columns are categorical, or use OneHotEncoder for those columns.

TRAIN AND TEST

Fit the encoder once, then reuse it

Split first

Fit OneHotEncoder on the training rows. Use that same fitted encoder to transform both the training and test rows.

Same learned columns

Train and test match

The encoder remembers the training categories, so both sets get the same feature columns.

New test category?

handle_unknown="ignore"

The encoder avoids an error. For that feature, the unknown category is represented by zeros.

Don’t fit the encoder on the test set; that lets test data influence preprocessing.

COLUMNTRANSFORMER

Different columns, different transformations

```
preprocess = ColumnTransformer([
    ("num", StandardScaler(),
     ["age", "hours-per-week"]),
    ("cat", OneHotEncoder(
        handle_unknown="ignore"),
     ["workclass", "education",
      "sex", "occupation"])
])

model = make_pipeline(
    preprocess,
    LogisticRegression(max_iter=1000)
)
```

Numeric columns

age and hours-per-week are standardized so they're on comparable scales.

Categorical columns

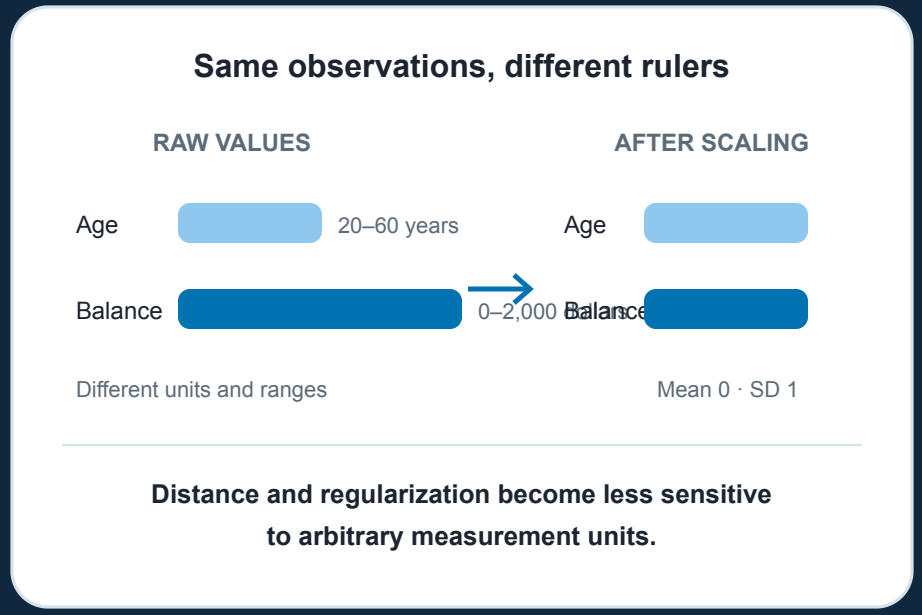
workclass, education, sex, and occupation are one-hot encoded.

The pipeline applies these steps before fitting the logistic regression model.

PART 2

Scaling

Put numeric features on comparable scales before modeling.



THE CANCER MEASUREMENTS

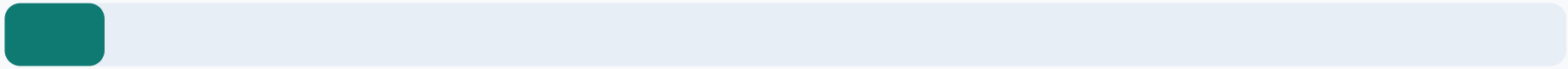
Area can dominate a distance calculation

Same person, two rulers

Mean area about 140 to 2,500



Mean smoothness about 0.05 to 0.16



KNN looks for nearby points. Area is so large that it decides who is nearby.

Without rescaling, the column with the biggest numbers dominates.

In KNN, features with larger numeric ranges can have more influence on distance.

THE FIVE WORKERS

Hours, measured in standard deviations

$$z = \frac{x - \mu}{\sigma}$$ (σ is the population standard deviation, as in StandardScaler)

The reference

Mean = 35.6 hours

For these five workers, the population SD is about 11.46 hours.

13 hours

$$z = \frac{13 - 35.6}{11.46} \approx -1.97$$

Nearly two standard deviations below the mean.

45 hours

$$z = \frac{45 - 35.6}{11.46} \approx 0.82$$

About eight-tenths of a standard deviation above the mean.

FOUR SCALING CHOICES

Choose what you want to preserve

StandardScaler

Mean 0, SD 1 per column

A common choice for scale-sensitive models. Outliers can still affect it.

MinMaxScaler

Map each column to 0–1

Useful for fixed-range data like pixels. Outliers can compress most values.

RobustScaler

Median and IQR per column

Less sensitive to outliers than scaling based on the mean and SD.

Normalizer

Scale each row to length 1

Use when the direction of a row matters more than its overall size.

SCALING DEPENDS ON THE MODEL

Trees use thresholds, not distances

Decision trees

“Is this value above the cut?”

Converting centimeters to inches keeps the same order of observations. The tree can make the same split with a converted threshold.

Scaling usually isn’t needed.

Scale-sensitive models

“How far apart are these values?”

KNN uses distances. Regularized logistic regression and neural networks can also be affected by feature scale.

Scaling can make a real difference.

KNN ON THE CANCER DATA

Scaling improves KNN on this split

89.5%

Test accuracy · no scaling

KNN compares distances using the original feature scales.

96.5%

Test accuracy · StandardScaler

Same model and split; the scaler is the only change.

One split is only one result

Another split showed a smaller gain. Use cross-validation to check whether scaling helps consistently, fitting the scaler within each training fold.

IN CODE

The scaler travels with the model

```
model = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier()
)

model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

Fit on training data

Transform both sets the same way

During fit, the scaler learns the training data's mean and spread. When you predict, it uses those same values to scale the test rows before sending them to KNN.

NO LEAKAGE

Fit on train. Transform test.

The scaler learns on the training rows only

Training data

- fit: learn the mean and the spread
- transform the training rows
- then fit the model

Test or validation

- transform with the same mean and spread
- then predict

The test set must not influence the scaler’s mean or standard deviation.

THE RULE

Fit preprocessing inside each training fold

Training fold

Fit each step here

Learn imputation values, category names, scaling statistics, selected features, and model parameters from these rows.

Validation fold

Transform, then evaluate

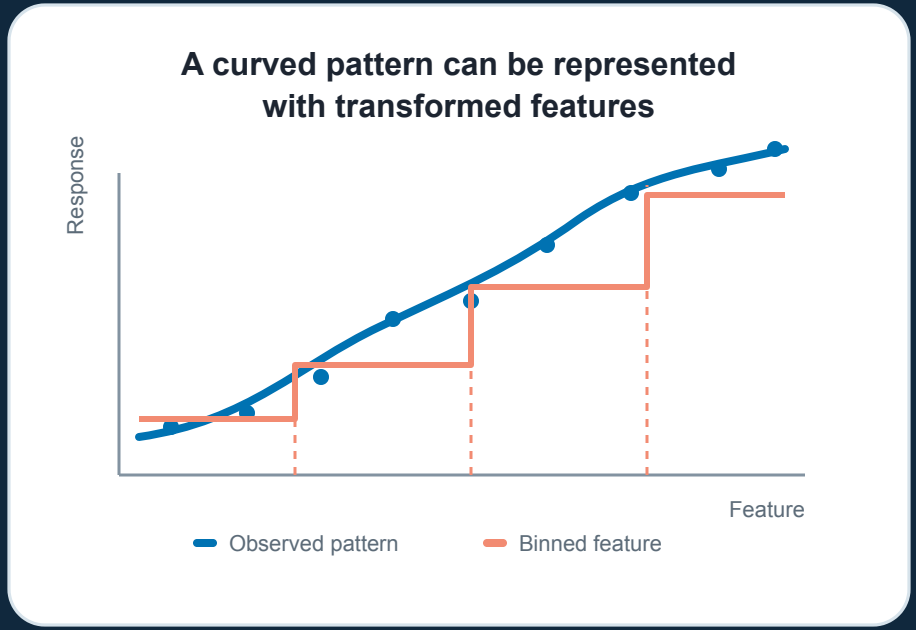
Apply the steps learned from the training fold. Do not refit them using validation rows.

Repeat this process for every cross-validation split.

PART 3

Transformations

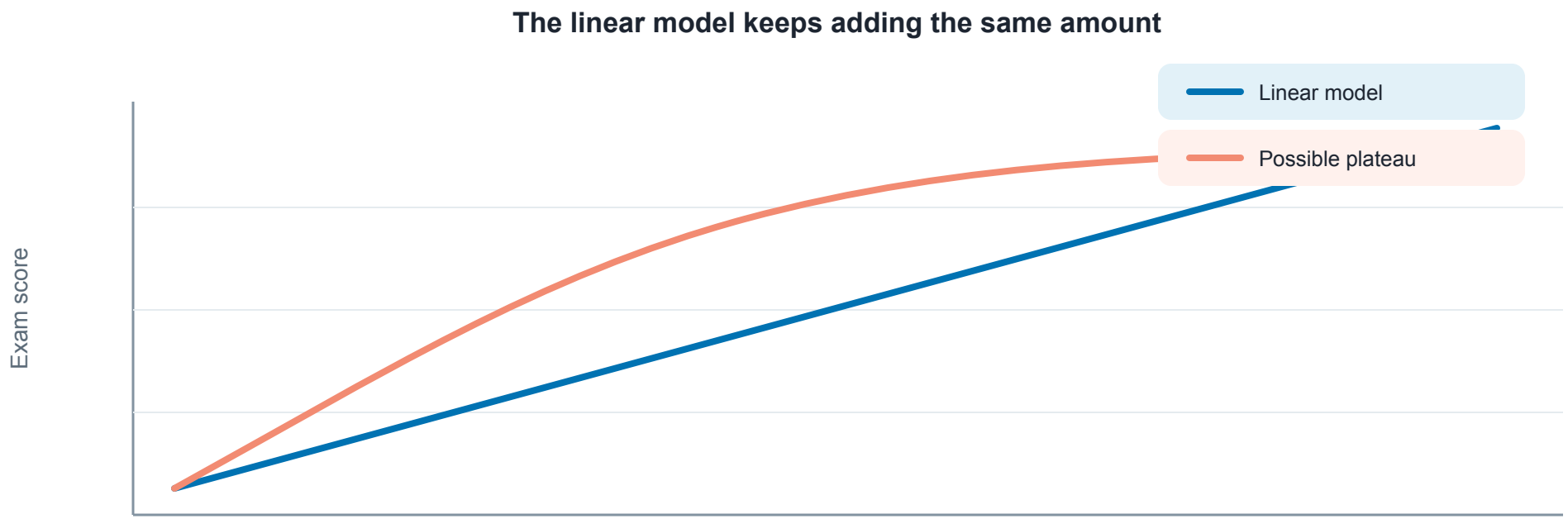
Reshape a feature so a linear model can capture nonlinear patterns.



STUDY HOURS AND AN EXAM SCORE

A straight line assumes the same gain per hour

$$\text{Score} = \beta_0 + \beta_1 \times \text{Hours}$$ The slope β_1 is constant.



A transformation can help a linear model represent a curved pattern.

TWO MORE TRANSFORMS

Let an effect bend—or depend on another feature

Hours and Hours²

Allow a curved relationship

Include both Hours and Hours². The squared term lets the effect bend; a negative coefficient can model gains that flatten over the range.

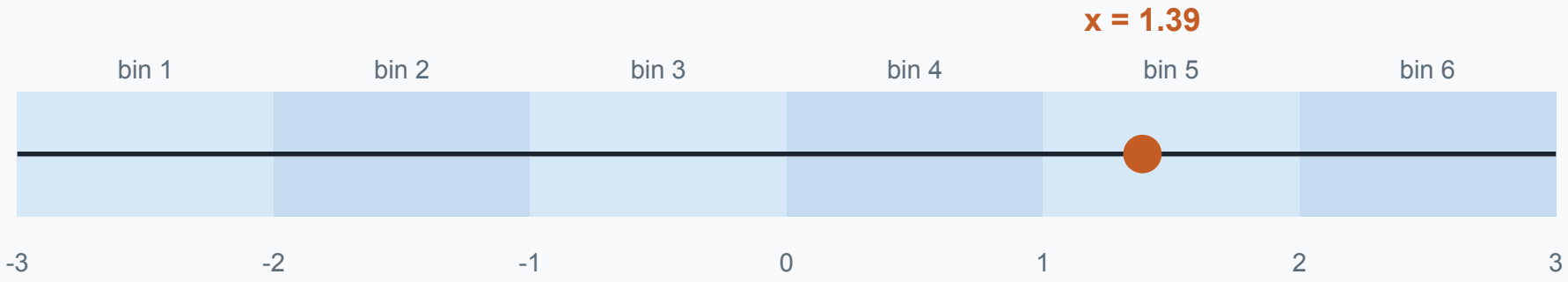
Hours × education

Let the effect vary by group

Encode education first, then interact Hours with its indicator columns. The model can learn a different Hours effect for each education group.

Replace exact values with intervals

Six equal intervals, from -3 to 3



Bin 5 turns on. The model receives 0 0 0 0 1 0.

A line had one slope for the whole range.

After binning, each interval can have its own level.

Binning keeps the interval information and discards differences between values in the same bin.

WHEN TO BIN

Binning adds steps, not slopes

A stepwise fit

One predicted level per bin

With one-hot bin indicators, predictions stay constant within each interval. Values in the same bin become indistinguishable.

Bin with a reason

Meaningful ranges or thresholds

Binning can capture changes across ranges or make a pattern easier to explain. Choose cut points using training data, then validate the result.

A LONG RIGHT TAIL

A log scale compresses large values

Website visits: 1, 2, 5, 10, 100, 1000

Before. 1, 2, 5, and 10 sit on top of each other.



After log base 10. Those four dots spread out.



From 10 visits to 1000 is a gap of 990. After the log, that gap is 2.

The log keeps the order but changes the distances: equal ratios become equal steps.

WHY A LOG HELPS

Compress large values; check the relationship

What the log does

Shrinks gaps among large values

For a right-skewed count such as logins or orders, a log transform compresses the high end while preserving the order.

When it can help

When it makes the pattern simpler

A log can make a curved relationship closer to linear. Check the pattern and validate the model; skew alone doesn’t guarantee a log will help.

For values that include zero, use log1p(x) = log(1 + x).

THE TOOLKIT

Six ways to shape what a model learns

01 Categories



One-hot encode labels. Don’t invent an order.

02 Target



Keep the answer out of the feature columns.

03 Scale



Put numeric features on comparable scales for KNN.

04 Folds



Fit preprocessing inside each training fold.

05 Bins



One-hot bins give each interval its own prediction level.

06 Logs



Compress large positive values; use log1p when zeros occur.

READING GUIDE

Preprocessing, features, and pipelines

Müller and Guido · Chapters 3, 4, and 6

CHAPTER 3

Scaling

Put numeric features on comparable scales.

CHAPTER 4

Encoding & transforms

Represent categories and reshape numeric features.

CHAPTER 6

Pipelines

Chain preprocessing steps with the model.

Scale → Encode and transform → Put the steps in a pipeline